

MISSÃO ARES-VII

Sistema de Monitoramento em Órbita Marciana

Global Solution 2026 · FIAP

Relatório Técnico — Análise, Estruturas, Lógica e Decisões de Projeto

Sumário Executivo

Este relatório documenta o desenvolvimento do sistema de monitoramento da missão experimental **ARES-VII**, projeto final da disciplina de Computational Thinking (Global Solution 2026, FIAP). O sistema opera em ambiente marciano com restrições severas: sem dependências externas além da biblioteca padrão Python 3, com fallback de dados embutidos e diagnóstico autônomo em tempo real. O documento está organizado em cinco eixos: (1) análise do problema e do cenário, (2) escolha e justificativa das estruturas de dados, (3) lógica booleana de diagnóstico, (4) previsão por regressão linear e (5) decisões técnicas de implementação.

Missão	ARES-VII — Órbita Marciana Experimental
Ciclo analisado	Ciclo 6 / Dia 3 (02:00 – 12:00)
Status final	CRÍTICO
Linguagem	Python 3.8+ (sem dependências externas)
Disciplina	Computational Thinking with Python — FIAP

1. Análise do Problema e Contexto da Missão

1.1 Cenário e desafios operacionais

A missão ARES-VII opera em órbita marciana sob condições radicalmente distintas das encontradas em sistemas terrestres. Três restrições fundamentais moldaram cada decisão de projeto: **radiação ionizante elevada** — que pode comprometer sensores e eletrônica —, **geração de energia intermitente** — dependente de ciclos solares e geração eólica marciana — e **latência de comunicação** com a Terra, que torna inviável a intervenção humana em tempo real. O sistema precisa operar de forma autônoma, detectar anomalias e emitir recomendações sem aguardar confirmação do Centro de Controle.

1.2 Evolução temporal dos parâmetros críticos

A tabela abaixo apresenta a evolução das variáveis principais ao longo do ciclo. A degradação é consistente e acelerada: a reserva energética cai de 72% para 32% em dez horas, enquanto a radiação quase dobra (35 → 78 mSv/h) e o sinal de comunicação colapsa de 85% para 15%. O consumo médio de 76,3 kWh supera consistentemente a geração combinada solar + eólica (média de 33,6 kWh), explicando o declínio energético estrutural observado.

Horário	Reserva (%)	Radiação (mSv/h)	Com. (%)	Vento (km/h)	Consumo (kWh)
02:00	72.0	35.0	85.0	45.0	65.0
04:00	65.0	38.0	82.0	52.0	58.0
06:00	55.0	52.0	60.0	78.0	70.0
08:00	45.0	65.0	42.0	95.0	82.0
10:00	38.0	72.0	28.0	115.0	88.0
12:00	32.0	78.0	15.0	130.0	95.0

Tabela 1 — Telemetria consolidada. Vermelho = crítico · Laranja = alerta · Verde = normal.

1.3 O cenário composto de falhas e a inconsistência de dados

O aspecto mais desafiador do Ciclo 6 é a **combinação simultânea de múltiplos sistemas em estado crítico**: a tempestade magnética que degrada a comunicação também intensifica a radiação; os ventos extremos que forçam os painéis ao modo de segurança reduzem a geração solar, acelerando o declínio energético. O sistema foi projetado para priorizar alertas em cenários compostos — não apenas detectar cada falha isoladamente.

Adicionalmente, o ciclo apresenta uma **inconsistência de dados** crítica: o módulo de comunicação reporta status binário 1 (OPERACIONAL), mas o sensor de qualidade de sinal registra apenas 15% — abaixo do limiar crítico de 20%. Essa divergência indica potencial falha de autodiagnóstico, possivelmente causada por bit-flip induzido por radiação no registrador de status do sensor. O sistema detecta e sinaliza essa inconsistência automaticamente, reforçando a importância da validação cruzada entre fontes independentes.

2. Estruturas de Dados — Escolha e Justificativa

A escolha das estruturas de dados foi guiada pelas **operações dominantes** que o sistema precisaria executar sobre cada conjunto de informações. A seguir, cada estrutura é apresentada com sua motivação técnica, complexidade operacional e trecho de código relevante.

2.1 Dicionário (hash table) — módulos da missão

O dicionário `modulos{}` mapeia o nome de cada módulo ao seu estado operacional. A escolha se justifica porque o sistema consulta módulos específicos dezenas de vezes por ciclo de diagnóstico. Em uma lista, essa busca teria complexidade $O(n)$; no dicionário, a tabela hash garante $O(1)$ no caso médio, independentemente do número de módulos.

```
modulos = {}
for m in lista_modulos:
    modulos[m['nome']] = {'status': int(m['status']), ...}
# Acesso: modulos['comunicacao']['status'] → O(1)
```

2.2 Listas — séries temporais de telemetria

As listas `historico_energia` e `leituras_ambientais` armazenam sequências ordenadas de leituras de telemetria. A lista preserva a ordem de inserção, permite iteração completa (necessária para a regressão linear) e oferece acesso $O(1)$ ao elemento mais recente via índice `-1` — padrão amplamente utilizado nas funções de diagnóstico, que sempre avaliam a leitura corrente. A escolha sobre deque se deve ao padrão de acesso sequencial e por índice, não exclusivamente pela extremidade.

2.3 Fila com prioridade (FIFO + sort) — sistema de alertas

A `fila_alertas` implementa o padrão FIFO para garantir que nenhum alerta seja descartado silenciosamente. Alertas chegam por ordem de detecção e são reordenados por prioridade após cada inserção. A decisão de não usar *heapq* foi intencional: a ordenação explícita por chave torna a lógica auditável e transparente — requisito crítico em sistemas de segurança de voo.

```
PRIORIDADE = {'CRITICO': 0, 'ALERTA': 1, 'NORMAL': 2}
def enfileirar_alerta(nivel, descricao, recomendacao):
    fila_alertas.append({'nivel': nivel, 'prioridade': PRIORIDADE[nivel], ...})
    fila_alertas.sort(key=lambda x: x['prioridade'])
```

2.4 Pilha (LIFO) — eventos críticos recentes

A `pilha_eventos` armazena exclusivamente eventos de nível CRÍTICO. O padrão LIFO permite acesso imediato ao evento crítico mais recente — o de maior relevância operacional — sem percorrer todo o histórico. Implementada sobre uma lista Python (`append` = push, exibição invertida = acesso ao topo), sem necessidade de classe separada.

2.5 Matriz e dicionário aninhado

A `matriz_leituras` (lista de listas) organiza os dados energéticos em formato tabular: cada linha é um horário, as colunas são variáveis (solar, eólica, consumo, reserva). O acesso duplo `matriz[linha][coluna]` permite análise cruzada sem dependências externas. Já a `hierarquia_missao` (dicionário aninhado) modela a árvore de dependência MISSÃO → {ENERGIA, HABITAT, OPERAÇÕES} → subsistemas, simulando um FMEA (Failure Mode and Effects Analysis) percorrível recursivamente.

Estrutura	Variável no código	Operação dominante	Complexidade
Dicionário	<code>modulos{}</code>	Leitura por chave (status)	$O(1)$ médio
Lista	<code>historico_energia</code>	Iteração completa (regressão)	$O(n)$
Fila + prioridade	<code>fila_alertas</code>	Inserção + reordenação	$O(n \log n)$
Pilha (LIFO)	<code>pilha_eventos</code>	Acesso ao topo (último crítico)	$O(1)$
Matriz	<code>matriz_leituras</code>	Acesso <code>[linha][coluna]</code>	$O(1)$
Dict. aninhado	<code>hierarquia_missao</code>	Varredura recursiva	$O(n)$

Tabela 2 — Resumo das estruturas de dados e suas características operacionais.

3. Lógica Booleana de Diagnóstico

3.1 Arquitetura em três níveis

O sistema classifica o estado de cada subsistema em três níveis discretos: NORMAL, ALERTA e CRÍTICO. Os limiares estão centralizados na estrutura global **LIMITES** — essa separação entre parâmetros e lógica é uma decisão de manutenibilidade: alterar limites de segurança não exige tocar no código de diagnóstico. O diagnóstico geral agrega os resultados individuais com operadores AND, OR e NOT aplicados explicitamente.

3.2 Expressão booleana formal

```
# CRÍTICO: qualquer uma das condições dispara o nível máximo
CRITICO = (energia == CRITICO) OR (NOT suporte_vida_ok)
          OR (comunicacao == CRITICO) OR (radiacao == CRITICO)

# ALERTA: nenhuma condição crítica, mas ao menos uma de alerta
ALERTA = NOT CRITICO AND (
  (energia == ALERTA) OR (NOT laboratorio_ok) OR
  (radiacao == ALERTA) OR (vento IN [ALERTA, CRITICO]) OR
  (comunicacao == ALERTA)
)

NORMAL = NOT CRITICO AND NOT ALERTA
```

No ciclo analisado, a condição CRÍTICO é verdadeira às 12:00 por quatro razões simultâneas: reserva energética em 32% com geração insuficiente, radiação acima de 75 mSv/h, qualidade de comunicação abaixo de 20%, e vento acima de 120 km/h.

3.3 Detecção de inconsistência de dados

A função *avaliar_comunicacao()* cruza o valor binário *status* do módulo (autodiagnóstico interno do hardware) com o valor métrico *qualidade_com* (medição independente do receptor). A inconsistência é sinalizada quando:

```
inconsistencia = (status_mod == 1) AND (qualidade < LIMITES['qualidade_com_critica'])
# status_mod=1 → módulo reporta OPERACIONAL
# qualidade=15% → abaixo do limiar crítico de 20%
# Contradição detectada: dispara alerta CRÍTICO específico
```

Esse padrão — **status OK + métrica crítica** — é um indicador clássico de falha silenciosa em sistemas embarcados, frequentemente causada por bit-flip induzido por radiação. Sem essa verificação cruzada, o diagnóstico geral poderia subestimar a gravidade da situação de comunicação.

Parâmetro	Limiar ALERTA	Limiar CRÍTICO	Unidade
Reserva energética	< 40%	< 20%	%
Consumo	—	> 90	kWh
Radiação	≥ 50	≥ 75	mSv/h
Qualidade com.	< 40%	< 20%	%
Vento	≥ 80	≥ 120	km/h

Tabela 3 — Limiares de segurança operacional (estrutura LIMITES).

4. Técnica de Previsão — Regressão Linear Manual

4.1 Justificativa da abordagem

A regressão linear por mínimos quadrados foi escolhida por três critérios: (1) **interpretabilidade** — o coeficiente angular m informa diretamente a taxa de queda em %/ciclo, legível pelo operador sem interpretação adicional; (2) **implementabilidade sem dependências** — o cálculo requer apenas aritmética básica, sem Numpy ou Pandas; (3) **adequação ao padrão dos dados** — a série 72%, 65%, 55%, 45%, 38%, 32% apresenta declínio aproximadamente linear.

4.2 Implementação OLS manual

```
def regressao_linear(x_vals, y_vals):
    n = len(x_vals)
    soma_x = sum(x_vals)
    soma_y = sum(y_vals)
    soma_xy = sum(x_vals[i] * y_vals[i] for i in range(n))
    soma_x2 = sum(x * x for x in x_vals)
    denom = n * soma_x2 - soma_x ** 2
    m = (n * soma_xy - soma_x * soma_y) / denom
    b = (soma_y - m * soma_x) / n
    return m, b  # y = m*x + b
```

Com os seis pontos do ciclo ($x = [0...5]$, $y = [72, 65, 55, 45, 38, 32]$), os coeficientes obtidos são $m \approx -8,114$ (queda de $\sim 8,1\%$ por ciclo de medição) e $b \approx 73,267$.

Horizonte	Índice x	Reserva prevista (%)	Situação
Ciclo +1	6	24.6%	ALERTA
Ciclo +2	7	16.5%	CRÍTICO
Ciclo +3	8	8.4%	CRÍTICO

Tabela 4 — Previsão energética por regressão linear ($m = -8,114 \cdot b = 73,267$).

4.3 Influência nas decisões e limitações do modelo

A previsão não é apenas informativa: quando o modelo indica que a reserva atingirá 20% em menos de 2 ciclos, uma recomendação de nível CRÍTICO é gerada automaticamente em `gerar_recomendacoes()` — mesmo que a reserva atual ainda esteja acima do limiar. Isso implementa um **alerta preditivo**, dando à equipe de controle tempo para agir antes da falha. O tempo estimado até o nível crítico é de aproximadamente 1,5 ciclos.

Limitação principal: o modelo linear assume taxa de declínio constante. Um aumento na geração solar ou a desativação do laboratório poderiam alterar a inclinação da reta. Em produção, recomenda-se recalibração a cada ciclo com janela deslizante (rolling window). Para o contexto de demonstração do Global Solution, o modelo é suficiente e satisfaz os requisitos da disciplina.

5. Decisões Técnicas de Implementação

5.1 Sem dependências externas

A restrição de não usar bibliotecas externas foi tratada como requisito de design. Em sistemas embarcados críticos — incluindo computadores de bordo de missões espaciais reais —, dependências externas introduzem risco de incompatibilidade e aumento de footprint de memória. Implementar regressão linear, estruturas de dados e diagnóstico booleano do zero demonstra que algoritmos fundamentais são suficientes para tomada de decisão em ambientes com recursos restritos.

5.2 Fallback de dados embutidos

A função `dados_embutidos()` fornece um conjunto de dados completo hardcoded no código. Se o arquivo JSON não for encontrado — por falha de I/O ou corrupção de armazenamento —, o sistema continua operando sem interrupção. Esse padrão de **graceful degradation** é um princípio de engenharia de sistemas críticos: a falha de um componente auxiliar não deve derrubar o sistema principal.

5.3 Arquitetura em camadas e design de alertas

O código segue uma arquitetura em camadas implícita: (1) carregamento e validação dos dados JSON, (2) inicialização das estruturas, (3) lógica de diagnóstico e geração de alertas, (4) análise preditiva, (5) renderização. Cada camada é implementada em funções independentes, sem acoplamento direto — o que permite substituir a camada de apresentação sem alterar nenhuma lógica de negócio.

Cada alerta carrega quatro campos: nível de prioridade numérico, descrição quantitativa com o valor medido, recomendação de ação específica, e o label textual. A combinação **valor medido + recomendação** é intencional: em situações de estresse operacional, o operador não deve precisar consultar outra tela para saber o que fazer.

5.4 Uso da IA no desenvolvimento

A ferramenta Claude (Anthropic) foi utilizada em quatro frentes: geração dos dados de telemetria simulados (com a inconsistência proposital), sugestão da organização modular do código, formulação das equações de mínimos quadrados e estruturação das expressões booleanas. Em todos os casos, a equipe realizou validação manual: os cálculos de regressão foram verificados à mão, cada regra booleana foi testada com casos extremos, e a inconsistência de comunicação foi confirmada como detectável na execução real do código.

6. Conclusão

O sistema de monitoramento da missão ARES-VII demonstrou que estruturas de dados clássicas, corretamente escolhidas, são suficientes para construir um sistema de diagnóstico robusto para ambientes críticos. A eficiência de acesso do dicionário, a ordenação natural da lista, a garantia de processamento da fila e o acesso imediato ao topo da pilha compõem um conjunto coeso e funcional — sem recorrer a frameworks ou bibliotecas de terceiros.

A lógica booleana em três níveis demonstrou capacidade de capturar cenários compostos de falhas simultâneas, e a detecção da inconsistência no módulo de comunicação reforça o valor da validação cruzada entre fontes independentes — princípio fundamental em engenharia de sistemas tolerantes a falhas.

A regressão linear manual, implementada sem dependências externas, transformou dados históricos em alerta preditivo: ao identificar que a reserva energética atingirá nível crítico em 1,5 ciclos, o sistema antecipa a falha em vez de apenas reagir a ela — capacidade essencial num ambiente onde a latência de comunicação com a Terra supera 20 minutos.